Enseignante : Mouna Maâtouk **1ér Licence Informatique de gestion**

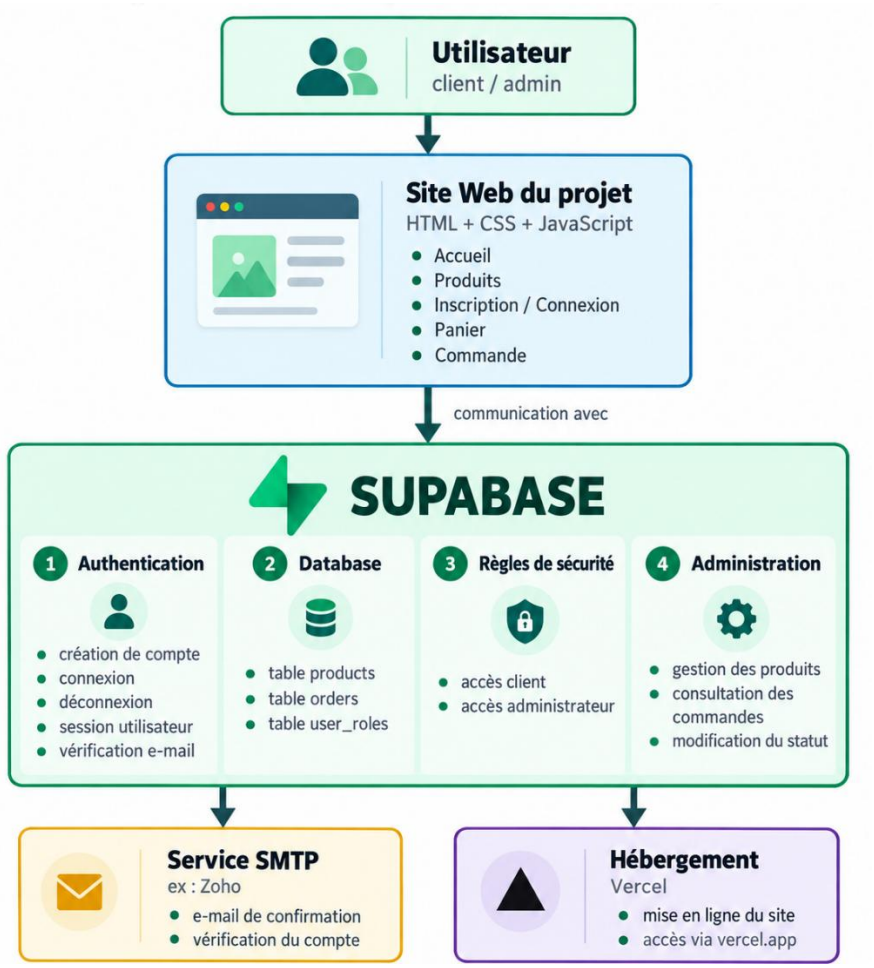
TP 3

Connecter son site à Supabase et le publier sur Vercel

Description du TP

Ce TP transforme un site vitrine statique en **boutique en ligne réellement connectée à une base de données**. Vous partez d'un site HTML, CSS et JavaScript que vous avez déjà construit (catalogue et, idéalement, panier) et vous lui ajoutez, avec l'aide d'un agent IA de code, un vrai système de comptes utilisateurs, un catalogue stocké en base, des commandes enregistrées et une interface d'administration.

Vous apprenez à répartir les rôles entre les services : **Supabase** héberge l'authentification et la base de données, **Vercel** héberge le site publié sur une adresse HTTPS. Vous découvrez aussi pourquoi la sécurité se joue dans la base (permissions et RLS) et non dans le navigateur, et comment contrôler le travail d'un agent au lieu de lui faire une confiance aveugle.



Le paiement reste une **simulation** : aucun débit et aucune donnée bancaire ne sont collectés.

Public	Étudiants disposant déjà d'un site HTML/CSS/JavaScript avec un catalogue et, idéalement, un panier.
Prérequis	Notions de HTML, CSS et JavaScript ; un éditeur de code avec agent IA ; comptes personnels Supabase, GitHub et Vercel ; une adresse e-mail consultable.
Outils	Éditeur avec agent IA, navigateur, Supabase (Auth, SQL Editor), GitHub, Vercel.
Compétences visées	Authentification et sessions ; modélisation SQL ; permissions et RLS ; intégration d'une API depuis le navigateur ; déploiement statique ; vérification critique du code produit par un agent.
Livrables	Le dossier du projet complet, le fichier `SUPABASE.md`, et l'URL du site déployé en `vercel.app`.

Objectifs et résultat attendu

À la fin, votre site permet de créer un compte, confirmer une adresse e-mail, se connecter, passer une commande de démonstration et consulter son historique. Un administrateur peut gérer les produits et consulter toutes les commandes pour modifier leur statut. Le site est accessible sur une adresse HTTPS en `vercel.app`.

Élément	Emplacement	Rôle
HTML, CSS, JavaScript et images	Votre dossier, puis Vercel	Présenter le site et les formulaires
Supabase Auth	Supabase	Gérer les comptes, mots de passe et sessions
`products`	Base Supabase	Stocker le catalogue
`user_roles`	Base Supabase	Identifier les administrateurs
`orders`	Base Supabase	Conserver les commandes
Panier	Navigateur	Conserver les articles avant validation
Paiement	Simulation	Aucun débit et aucune donnée bancaire

> **À retenir :** Vercel héberge le site ; Supabase héberge l'authentification et la base. Un fichier SQL déposé dans le projet n'est pas exécuté automatiquement par le navigateur ou par Vercel.

Les noms de pages ci-dessous sont ceux du TP ; conservez les équivalents de votre projet si leurs noms diffèrent. Documentation en ligne vérifiée le 12 octobre 2026 ; les intitulés des tableaux de bord peuvent évoluer.

1. Préparer votre projet et votre agent

1.1. Sauvegarder le travail existant

1. Créez une copie du projet ou un commit si vous utilisez Git.
2. Repérez le dossier contenant `index.html`.
3. Vérifiez que les images et les liens fonctionnent avant l'intégration.
4. Notez vos catégories, devise et frais de livraison.
5. Préparez une adresse e-mail que vous pouvez consulter.

Chaque étudiant utilise son propre projet Supabase. Ne réutilisez pas l'URL ou la clé d'EcoBox.

1.2. Donner l'ordre à l'agent

1. Ouvrez votre dossier dans votre éditeur avec l'agent.
2. Ouvrez le fichier joint `PROMPT-supabase.txt`.
3. Complétez la section « Mon contexte ».
4. Copiez le prompt complet dans l'agent.
5. Laissez-le inspecter puis modifier votre projet.
6. Relisez les fichiers livrés et les actions manuelles annoncées.

L'agent prépare le code et les scripts. Vous réalisez les opérations dans les tableaux de bord Supabase, GitHub et Vercel avec vos propres comptes.

1.3. Contrôler les fichiers attendus

```
mon-projet/
├── index.html
├── produits.html           ou votre catalogue existant
├── signup.html             inscription
├── login.html              connexion
├── verification.html       instruction de confirmation d'e-mail
├── compte.html             identité et historique
├── panier.html
├── checkout.html           validation de commande
├── confirmation.html
├── admin.html              produits et commandes
├── style.css
├── script.js
├── supabase-client.js      URL + clé publique + client
├── supabase-shop.js        appels Supabase
├── supabase-setup.sql      catalogue + rôles + droits
├── supabase-admin.sql      attribution manuelle du rôle admin
├── supabase-orders.sql     commandes + fonction + droits
├── SUPABASE.md             instructions adaptées au projet
└── images/
```

Les `.sql` sont des scripts pour la base ; ce ne sont pas des pages du site. Si votre projet utilise déjà des dossiers `js/` ou `css/`, adaptez les chemins sans déplacer inutilement les fichiers.

2. Créer le projet Supabase

2.1. Créer la base distante

1. Ouvrez supabase.com et connectez-vous.

2. Depuis le tableau de bord, choisissez **New project**.
3. Sélectionnez ou créez votre organisation.
4. Saisissez un nom reconnaissable, par exemple `boutique-prenom`.
5. Définissez le mot de passe de la base et conservez-le dans un emplacement privé.
6. Choisissez la région proposée adaptée à votre localisation.
7. Créez le projet et attendez la fin de sa préparation.

Le mot de passe de la base ne doit jamais être placé dans `supabase-client.js`. Ce n'est pas non plus le mot de passe de votre futur compte client du site.

2.2. Trouver l'URL et la bonne clé

1. Dans votre projet, ouvrez **Connect**.
2. Repérez **Project URL** : elle ressemble à `https://xxxxxxxx.supabase.co`.
3. Copiez la **Publishable key**, généralement préfixée par `sb_publishable_`.
4. Pour retrouver les clés, utilisez également **Settings → API Keys**.

La clé publishable est prévue pour le navigateur ; les données sont protégées par les permissions et RLS.

> **Attention :** ne copiez jamais une clé `secret`, `sb_secret_…` ou `service_role` dans le site. Documentation : supabase.com/docs/guides/getting-started/api-keys.

3. Relier les fichiers JavaScript

Le site n'a pas d'étape de compilation : les scripts sont chargés directement par le navigateur. Vérifiez dans chaque page concernée l'ordre de chargement, avec l'attribut `defer`.

1. Bibliothèque Supabase JavaScript v2.
2. `supabase-client.js` : une seule initialisation de `supabase.createClient` avec la Project URL et la Publishable key.
3. `supabase-shop.js` : authentification, catalogue, commandes et administration.
4. `script.js` : raccordement au panier et aux interfaces existantes.

```
<script defer src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
<script defer src="supabase-client.js"></script>
<script defer src="supabase-shop.js"></script>
<script defer src="script.js"></script>
```

Remplacez dans `supabase-client.js` les valeurs d'exemple par celles de votre projet (étape 2.2). Évitez toute double initialisation du client.

```
const SUPABASE_URL = "https://VOTRE-PROJET.supabase.co";
const SUPABASE_KEY = "sb_publishable_VOTRE_CLE";
const sb = supabase.createClient(SUPABASE_URL, SUPABASE_KEY);
```

N'utilisez pas `process.env` dans un script livré au navigateur : un fichier `.env` n'est pas lu automatiquement par un site statique. N'ajoutez ni backend ni outil de compilation inutile.

4. Exécuter les scripts SQL

Les scripts préparés par l'agent doivent être exécutés **par vous**, dans le **SQL Editor** de votre projet Supabase. Respectez l'ordre ci-dessous.

Ordre	Fichier	Contenu et précaution
1	`supabase-setup.sql`	Tables `products` et `user_roles`, contraintes, permissions, RLS, données initiales. Réservé à une base neuve.
2	`supabase-orders.sql`	Table `orders`, permissions, RLS et fonction `create_demo_order`. Relançable sans supprimer les commandes.
3	`supabase-admin.sql`	Attribution du rôle `admin` à un compte existant. À exécuter après avoir créé et confirmé votre compte (étape 5).

1. Ouvrez **SQL Editor** dans Supabase et créez une nouvelle requête.
2. Collez le contenu de `supabase-setup.sql`, puis exécutez-le.
3. Faites de même avec `supabase-orders.sql`.
4. Dans **Table Editor**, vérifiez la présence de `products`, `user_roles` et `orders`, avec RLS activé.

> **Sécurité :** ne désactivez jamais RLS pour faire disparaître une erreur. Les montants des commandes sont calculés par la base à partir du catalogue ; le navigateur n'est jamais cru sur un prix ou un propriétaire.

5. Configurer l'authentification

5.1. Déclarer les adresses du site

1. Dans Supabase, ouvrez **Authentication**, puis la section de configuration des URL (**URL Configuration**).
2. Renseignez l'adresse locale de test (par exemple `http://localhost:8000`) dans les URL de redirection autorisées.
3. Après le déploiement, ajoutez l'adresse `https://votre-projet.vercel.app` et mettez à jour la **Site URL**.

5.2. Créer et confirmer un compte

1. Ouvrez `signup.html` sur votre site local et créez un compte avec une adresse que vous pouvez consulter.
2. Ouvrez l'e-mail de confirmation et cliquez sur le lien. Pensez à vérifier les courriers indésirables.
3. Connectez-vous avec `login.html` et vérifiez que `compte.html` affiche votre identité.

5.3. Devenir administrateur

1. Dans Supabase, ouvrez **Authentication → Users** et copiez l'**UUID** de votre compte.
2. Ouvrez `supabase-admin.sql`, remplacez `VOTRE-UUID` par cet identifiant.
3. Exécutez le script dans **SQL Editor**.
4. Déconnectez-vous puis reconnectez-vous : le lien **Admin** doit apparaître.

Cette attribution se fait uniquement dans SQL Editor, jamais depuis le navigateur.

6. Tester le parcours complet

Masquer un bouton ne constitue pas un test de sécurité. Testez avec des sessions réelles, et pas seulement avec SQL Editor, qui est connecté en propriétaire et peut contourner RLS.

Profil	Vérifications attendues
Visiteur	Voit uniquement les produits disponibles ; ne peut lire aucune commande ; est invité à se connecter avant de commander.
Client A	Passe une commande de démonstration ; la retrouve dans son historique ; le panier est vidé seulement après succès ; ne voit pas la page admin.
Client B	Ne voit jamais les commandes du client A ; ne peut modifier ni rôle, ni prix, ni statut.
Admin	Ajoute, modifie et supprime un produit (avec confirmation) ; consulte toutes les commandes ; change un statut ; un changement concurrent est signalé.

Testez aussi les cas d'erreur : coupure réseau pendant la commande (aucun doublon), formulaire invalide, liste vide. Ne créez pas de commande réelle et n'effacez aucune donnée existante pour tester.

7. Publier sur Vercel

1. Poussez votre projet sur un dépôt GitHub personnel.
2. Dans Vercel, choisissez **Add New → Project** et importez ce dépôt.
3. Vérifiez le **Root Directory** : il doit désigner le dossier contenant `index.html`.
4. Choisissez le preset **Other** (site HTML statique). Laissez vide la commande de build ; adaptez le dossier de sortie si besoin.
5. Lancez le déploiement et notez l'adresse `https://…vercel.app`.
6. Retournez dans Supabase et ajoutez cette adresse aux URL d'authentification (étape 5.1).
7. Refaites les tests de l'étape 6 sur l'URL HTTPS finale.

Si votre projet utilise un framework, adaptez ces réglages à sa vraie compilation.

8. Erreurs fréquentes

Symptôme	Piste de résolution
`supabase is not defined`	La bibliothèque Supabase n'est pas chargée avant `supabase-client.js`, ou le chemin est faux.
Le catalogue reste vide	Script `supabase-setup.sql` non exécuté, produits non marqués disponibles, ou politique RLS manquante.
L'e-mail de confirmation n'arrive pas	Vérifier les indésirables, l'adresse saisie, puis patienter ; le service d'e-mail de test a des limites d'envoi.
Le lien de confirmation redirige vers une mauvaise adresse	Mettre à jour la Site URL et les URL de redirection dans Supabase.
Le lien Admin n'apparaît pas	UUID incorrect dans `supabase-admin.sql`, script non exécuté, ou session à rafraîchir (déconnexion/reconnexion).
Erreur à la création de commande	Utilisateur non connecté, `supabase-orders.sql` non exécuté, ou panier contenant un produit indisponible.
Page blanche sur Vercel	Mauvais Root Directory ou dossier de sortie ; vérifier la présence de `index.html` à la racine déployée.

9. Livrables et critères de validation

- Le dossier complet du projet, avec les fichiers listés au point 1.3.
- `SUPABASE.md` complété et adapté à votre projet.
- L'URL de votre site en `vercel.app`.
- Aucune clé `secret` ou `service_role`, ni mot de passe de base, dans le code.
- Les quatre profils de test de l'étape 6 validés.

Pensez à relire ce que l'agent annonce avoir fait : il ne peut pas affirmer avoir exécuté le SQL dans votre Supabase ni déployé votre site à votre place.